

2. Another way of using directive.

```

• <template
•     ngFor #name="$implicit"
•     [ngForOf]="names">
•         <div [ngClass]="{'selected': name == selectedName,
• 'unselected' : name != selectedName}"
•         (click)="nameClicked($event)">{{name}}</div>
•     </template>

```

3. Build style class. Into the document these classes are injected automatically.

```

• styles : [ `
•     .selected {margin: 6px;padding: 6px;width :
100px;border: 2px solid black;color : #ffffff;background-
color: #ff0000;}
•     .unselected{margin: 6px;padding: 6px;width :
100px;border : none;color : #000000;background-color:
#ffffff;}` ]

```

4. Use of directives

```

• directives : [NgFor, NgClass]

```

5. Component class of application. Instance variables are created for list of names and selected names, which are bound to the html markup of the template.

```

• export class App :
•     names:Any = ['digit1','digit2','digit3','digit4'];
•     selectedname:String = 'digit1':

```

6. This method is called on click event on the name. Sets the selected name instance variable bound to the template.

```

• nameClicked(event:Event){
•     this.selectedName = event.srcElement.innerHTML.trim();

```

NgStyle

<http://pastebin.com/edit/AD5LaiWq>

For setting styles of an element this directive is used.

Simple code to add set one style:

```

• <div [style.fontSize]="selected? 'x-large': 'smaller'" >

```